

The main function will look like what follows:

```
int main (int argc, char** argv){
    glutInit(&argc, argv);
    // Initializing glut
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    // Setting up display mode, the single buffering and RGB colors
    glutInitWindowPosition(80, 80);
    // Position of the window on the screen
    glutInitWindowSize(400, 300);
    // Size of the window
    glutCreateWindow("A simple Triangle");
    // Name of the window
    glutDisplayFunc(display);
    // The display function
    glutMainLoop();
}
```

All the above are GLUT calls. Since OpenGL doesn't handle windowing calls, all the window creation is taken care of by GLUT. We initialise a window and set up the display mode. The display mode specified above states that the window should display RGB colours and will require a single buffer to handle pixel outputs. We then specify the window position on the screen, the size of the window to be displayed and the display function. The display function takes a pointer to a function as an argument. The function would describe what is to be displayed inside the screen. The final function call loops the main forever, so that the window is always open until closed.

Now that a window is initialised, let's draw a triangle with OpenGL function calls. Here's how the display function looks:

```
void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glBegin(GL_POLYGON);

    glColor3f(1, 0, 0);
    glVertex3f(-0.6, -0.75, 0.5);

    glColor3f(0, 1, 0);
    glVertex3f(0.6, -0.75, 0);

    glColor3f(0, 0, 1);
    glVertex3f(0, 0.75, 0);
}
```

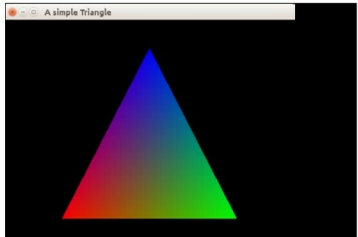


Figure 3: A simple triangle: C++ based OpenGL example program

```
glEnd();
glFlush();
}
```

All OpenGL based function calls begin with the letters *gl* in lowercase. We are planning to display a simple triangle defined by three vertices. We specify the colour and position of the vertices and map them on to the screen. For this, we first call the *glClear()* function which clears the display buffer. Next, we specify the type of primitive we are planning to draw with the *glBegin()* function. This is the primitive of our drawing until a *glEnd()* function is called. Next, between the *glBegin()* and *glEnd()* we draw three points (vertices) and form a triangle with them. We first have to specify the colour and then the position of the drawing. As OpenGL is platform independent, the C-style datatypes are specified as a function call. This can be seen with the *glColor3f()* function, which takes three float values.

Once the program is compiled, linked and called as described above, it would look like what's shown in Figure 3.

If you're interested in learning OpenGL based computer graphics, the old version of the official 'red book' is available for free at <http://www.glprogramming.com/red/>. 

By: Sricharan Chiruvolu

The author is a FOSS enthusiast, Web developer and a graphics programmer. He is a student at Amrita School of Engineering, Bengaluru, and a student developer for Copyleft Games in association with Google, under Google Summer of Code - 2015. You can contact him at sricharanizeo@gmail.com



For More

Test and Measurement Stories

Visit www.electronicsb2b.com